Glimm – Licht ist Leben

Konzeption und Umsetzung eines 2D-Metroidvania-Plattformers in Godot 4.7

Autor	Romed Kelmer	**Umfang**	6 Level inkl. Bosskampf, Tastatur- und Controllersteuerung
Lehrveranstaltung	Creative Game & XR Experience	**Repository**	[Link eintragen]
Engine / Sprache	Godot Engine 4.7 (Forward+), GDScript	**Stand**	Oktober 2026

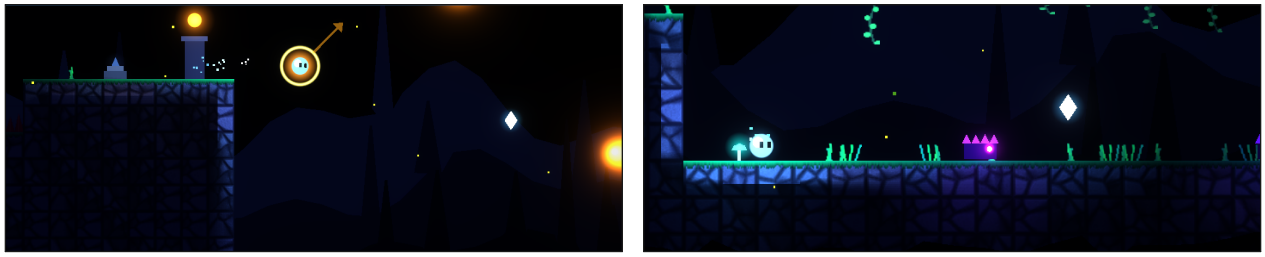


Abbildung 1: Links: Bash an einem Lichtsamen (Zeitlupe, Zielpfeil). Rechts: Level 3 mit Krabblern, Leuchtpilz und Lichtsplitter. Die Spielfigur ist die einzige Lichtquelle und wirft dynamische Schatten.

1 Game Design Sheet

1.1 Spielidee

Glimm ist ein kleiner Lichtgeist in einer dunklen Höhle und muss den *Lichtbaum* am Ende jedes Levels erreichen. Das Spiel ist von ***Ori and the Will of the Wisps*** (Moon Studios, 2020) inspiriert: Von dort stammen das flüssige Movement mit Doppelsprung, Walljump und Dash sowie die Idee des „Bash“, bei dem sich die Spielfigur an einem Objekt festhält, in Zeitlupe zielt und wegschleudert. Grafiken, Figuren und Namen aus Ori wurden nicht übernommen.

Kernidee

Licht ist zugleich **Sichtweite, Lebensenergie und Ressource**. Mit abnehmendem Licht schrumpft der sichtbare Bereich um die Spielfigur. Da jede Fähigkeit Licht verbraucht, entsteht eine fortlaufende Abwägung zwischen schnellem Vorankommen und dem Sparen von Licht für das nächste Lichttor.

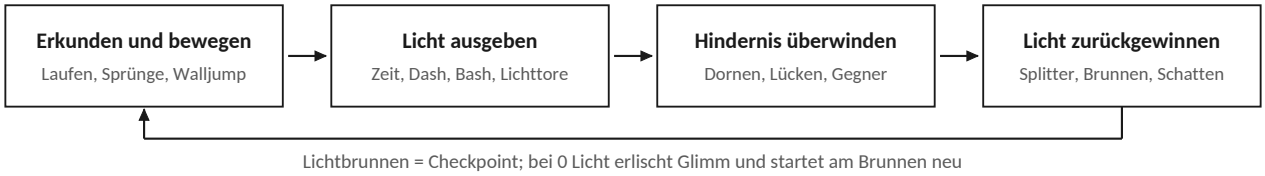
−1,2/s	−20	−10	−40/50	+35	+15/20	100
Licht verfällt ständig	Dash	Bash	Lichttor öffnen	Lichtsplitter	Schatten besiegt	Lichtbrunnen

Tabelle 1: Lichtökonomie (Werte in Lichtpunkten, Maximum 100). Bei 0 Licht erlischt die Spielfigur und startet am letzten Lichtbrunnen neu; die Bildvignette verstärkt sich mit sinkendem Lichtstand.

1.2 Zielgruppe

Spieler:innen ab etwa 12 Jahren, die präzise 2D-Plattformer mögen (z. B. *Ori*, *Celeste*, *Hollow Knight*), mit kurzen Spielsitzungen. Tutorial-Hinweise führen schrittweise an die Mechaniken heran. Controller (Xbox/PlayStation) und Tastatur werden vollständig unterstützt.

1.3 Core Gameplay Loop



Langfristige Schleife: Ein Schrein schaltet eine Fähigkeit frei, dadurch werden neue Wege erreichbar (Metroidvania).

Abbildung 2: Kurzfristige Schleife (Licht ausgeben und zurückgewinnen) und langfristige Metroidvania-Schleife (Fähigkeiten öffnen neue Wege).

1.4 Mechaniken, Gegner und Level

Element	Beschreibung
Bewegung	Variable Sprunghöhe, Coyote Time, Jump Buffer, Doppelsprung, Walljump (auch an einer einzelnen Wand)
Dash	Kurzer Sprint ohne Schwerkraft; mit Sprung abbrechbar (Dash-Jump behält Schwung); besiegt Krabbler
Bash	An Lichtsamen, Irrlicht oder Projektil festhalten → Zeitlupe → zielen → wegschleudern; frischt Doppelsprung und Dash auf
Schatten-Irrlicht	Schwebt oder patrouilliert, Berührung ist tödlich – aber bash-bar
Krabbler	Läuft am Boden, dreht an Kanten und Wänden um; nur per Dash besiegbar
Speier	Schießt bei Sichtkontakt Kugeln; eine zurückgebashte Kugel zerstört ihn
Schattenherz (Boss)	Schwebt über der Arena, 6 Leben, 3 Phasen: gezielte Kugel → Dreier-Fächer + Krabbler → Kugelring + Dunkelheit. Nur mit eigenen, zurückgebashten Kugeln besiegbar
Level	1 Wurzelhöhle (Tutorial, Dash/Bash) · 2 Der Schacht (vertikal) · 3 Schattenhain (Gegner-Parcours) · 4 Irrlichtschlucht (Bash-Ketten) · 5 Dunkle Tiefe (dunkler, schnellere Irrlichter) · 6 Herz der Schatten (alles kombiniert + Bosskampf). Steigende Torkosten und Irrlicht-Tempo, weniger Licht

1.5 Steuerung und Feedback

Aktion	Tastatur	Controller
Laufen / Zielen	A/D bzw. ←/→ · W/S zum Zielen	Linker Stick / D-Pad
Springen (halten = höher)	Leertaste oder J	A / Kreuz
Dash	Shift oder K	X / RB
Bash (halten, zielen, loslassen)	E oder L	Y / LB
Pause · Neustart	Esc/P · R	Start · Back

Hinweistexte zeigen automatisch die Tasten des zuletzt benutzten Geräts. Feedback entsteht durch Squash & Stretch, Partikel, Soundeffekte, Screenshake, Zeitlupe beim Bash, einen kurzen Farbversatz (Bash, Tod), eine flackernde Lichtleiste bei wenig Licht und einen Lichtkegel, dessen Radius der Lichtmenge entspricht.

2 Technische Architektur

2.1 Technologie

Umgesetzt mit **Godot 4.7** und statisch typisiertem **GDScript** (47 Skripte, rund 2.800 Zeilen, 28 Szenen). Gerendert wird mit **Forward+** (Vulkan) und aktiviertem HDR für 2D, damit Glow möglich ist. Godots Node- und Szenensystem wird bewusst genutzt: Alles Sichtbare besteht aus Nodes (Sprite2D, Polygon2D, Line2D, Label) und wird mit AnimationPlayer animiert, Skripte enthalten nur Logik. Kommunikation läuft über Signals, globale Zustände liegen nur in Autoloads.

2.2 Szenenstruktur

```
MainMenu (Control) ──► Main (Node)     Autoloads: InputHints · Events · GameState · Audio
Main
├─ WorldEnvironment          Glow / Farbanpassung
├─ Level  (aus resources/level_catalog.tres geladen)
│   ├─ Darkness (CanvasModulate) · Background (Parallax2D + Polygon2D)
│   ├─ Terrain (TileMapLayer, Terrain-Set „Fels“) · TerrainShadows (LightOccluder2D-Nodes)
│   └─ Hazards · Objects · Decor (Gras/Ranken/Pilze) · Enemies · Hints · Fireflies
├─ Player (CharacterBody2D)
│   ├─ Visual (Sprite2D/Polygon2D) · Effects (Partikel + Sounds) · Light · Camera
│   ├─ WallCheckLeft/Right (RayCast2D) · BashDetector (Area2D)
│   └─ StateMachine — Idle · Run · Jump · Fall · WallSlide · Dash · Bash · Dead
├─ Foreground · PostFx       CanvasLayer (Silhouetten, Vignetten-Shader)
└─ HUD · PauseMenu           CanvasLayer (UI, läuft auch pausiert)
```

2.3 Zentrale Klassen

Klasse / Datei	Aufgabe
Player	Gemeinsame Bausteine: Schwerkraft, Laufen, Timer, Fähigkeits-Checks; alle Werte als @export im Inspector justierbar
StateMachine, PlayerState	Endlicher Automat; jeder Zustand ist ein eigener Node. Basisklassen GroundedState und AirborneState vermeiden doppelten Code
GameState (Autoload)	Licht, Fähigkeiten, Checkpoint, Zeit, Tode; meldet Änderungen per Signal
LevelCatalog, LevelEntry	Eigene Resource-Klassen: Levelliste als .tres, im Inspector per Drag-and-drop bearbeitbar
Audio (Autoload-Szene)	Musik mit Überblendung, UI-Sounds, Low-Pass-Effekt auf dem Music-Bus im Pausenmenü
Events (Autoload)	Event-Bus für einmalige Ereignisse (Tod, Hinweis, Levelende, Screenshake)
InputHints (Autoload)	Erkennt Tastatur/Controller und ersetzt Platzhalter wie {jump} in Texten
BashOrb → ShadowWisp, ShadowBolt	Bash-Ziel als Basisklasse; Gegner erben die Bash-Logik und überschreiben nur Aussehen und Verhalten
Crawler, Spitter	Bodengegner (CharacterBody2D + RayCast-Kantencheck) und Geschützturm (Sichtlinie per RayCast2D)
ShadowHeart, BossArena	Boss meldet nur phase_changed/defeated; die Arena steuert Tor, Krabbler, Dunkelheit und Musik, das HUD hört über den Event-Bus zu. Treffer über die gemeinsame Methode take_reflected_hit()
Level, Hud, PauseMenu, PostFx	Level stellt Spawn und Weltgrenzen bereit; UI und Effekte reagieren nur auf Signale
AutoTerrain, TerrainShadows, Decor	@tool-Skripte im Editor: korrigieren Kachel-Varianten, erzeugen Schatten-Occluder beim Malen und verteilen Deko per Inspector-Button (siehe 2.5)

2.4 Entkopplung über Signals

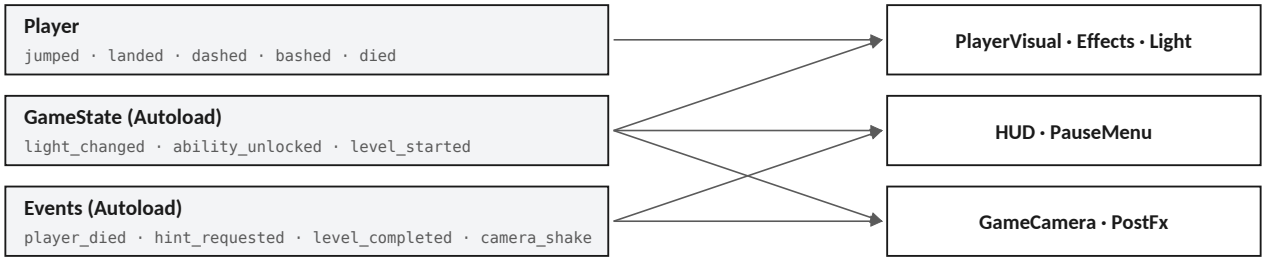


Abbildung 3: Signalfluss. Sender kennen ihre Empfänger nicht, sie senden lediglich das Signal.

Beispiel: Stirbt Glimm, wechselt die Zustandsmaschine in *Dead*. Der Player sendet *died* (Visual und Partikel reagieren), der Event-Bus sendet *player_died* (HUD blendet ab, PostFx erzeugt einen Farbversatz, die Kamera wackelt). Keiner dieser Empfänger ist im Player-Code erwähnt.

2.5 Alles im Editor: Level, Optik und Sound

Jedes Level ist eine normale Godot-Szene. Fels wird mit dem TileMap-Werkzeug gemalt – über das Terrain-Set „Fels“ wählt Godot automatisch Moos-, Wurzel- oder Steinkacheln. Objekte und Gegner werden als Szenen hineingezogen und im Inspector eingestellt (Schwebestrecke, Dornenbreite, Torkosten, Fähigkeit); @tool-Skripte zeigen diese Werte direkt im Editor an. Schatten sind gespeicherte *LightOccluder2D*-Nodes, die sich beim Malen neu aufbauen. Gras, Ranken und Leuchtpilze sind einzelne Szenen-Instanzen, die ein Inspector-Button (*@export_tool_button*) verteilt und die danach frei verschoben werden können. Neue Level entstehen aus `level_template.tscn` und werden im Level-Katalog eingetragen.

Optik: Glow über *WorldEnvironment* mit HDR-Farben über 1,0, dynamische Schatten, Normal Maps über *CanvasTexture*, Shader für Wind (Gras, Ranken), Lichttore und Post-Processing. Sound: Jede Szene trägt ihre eigenen *AudioStreamPlayer*-Nodes (z. B. Sprung mit *AudioStreamRandomizer* für Tonhöhen-Variation), Musik und Effekte laufen über getrennte Audio-Busse.

2.6 Ausführung – Schritt für Schritt

Anleitung

1. Godot 4.7 (Standard-Version, nicht .NET) von godotengine.org herunterladen und starten.
2. *Importieren* wählen und die Datei `project.godot` im Projektordner öffnen.
3. Mit **F5** starten und im Hauptmenü „Spielen“ oder ein Level aus der Liste wählen.
4. Falls Vulkan auf einem sehr alten Rechner fehlt: Projekteinstellungen → Rendering → Renderer auf „Mobile“.

3 Learnings & Challenges

3.1 Was gut funktioniert hat

- **Zustandsmaschine:** Dash, Bash und Walljump ließen sich als neue Zustände hinzufügen, ohne bestehenden Code umzubauen. Fehler waren klar einem Zustand zuzuordnen.
- **Signals und Event-Bus:** HUD, Pausenmenü und Post-Effekte kamen später dazu, ohne dass der Spieler-Code geändert werden musste.
- **Vererbung bei Gegnern:** Weil Irrlicht und Projektil von BashOrb erben, waren sie sofort bash-bar. Das Ori-typische „Projektil zurückschleudern“ kostete nur wenige Zeilen.
- **Editor-Workflow:** Weil alles aus Nodes, Szenen und Resources besteht, lassen sich Level, Aussehen, Animationen und Sounds ohne Code ändern. Abstände (z. B. eine Lücke, die nur mit Dash schaffbar ist) wurden zusätzlich mit automatisierten Testläufen (simulierten Eingaben) überprüft.
- **Licht als verbindendes Thema:** Gegner geben Licht zurück, Tore verlangen Licht, die Vignette zeigt den Lichtstand. Dadurch wirkt das Konzept geschlossen.

3.2 Hürden und Lösungen

Problem	Ursache	Lösung
Kachel-Varianten aktualisierten sich beim Malen nicht	Das Signal changed von TileMapLayer kommt bei Zellenänderungen nicht zuverlässig	Virtuelle Methode _update_cells() überschrieben; Update gesammelt per call_deferred
Bash-Halte-Limit zu lang	Engine.time_scale verlangsamt auch delta	Echtzeit = delta / time_scale; UI-Tweens ignorieren time_scale
Felsen komplett schwarz	Kachel-Occluder beschatten die Kachel selbst; flache Lichtstrahlen werden an Kachelkanten blockiert	TerrainShadows erzeugt zusammengefasste Occluder, an freien Seiten 20 px eingerückt
Krabbler zitterte an Kanten und fiel hinunter	Nach dem Umdrehen lieferte der Kanten-RayCast noch das Ergebnis der alten Seite	RayCast sofort versetzen und force_raycast_update() aufrufen
Boss wurde nie getroffen	Zurückgebashte Kugeln trafen Decke und Plattformen	Reflektierte Kugeln kollidieren nur mit Layer „enemies“ und lenken leicht zum Ziel (Gruppe reflect_target)
Fehler „flushing queries“ in Phase 2	Krabbler wurden im Physik-Callback des Treffers erzeugt	Erzeugung per call_deferred() nach dem Physik-Schritt

3.3 Kritische Reflexion

- **Audio:** Die Soundeffekte sind synthetisch erzeugt und eher schlicht. Lautstärken und Mischung sind nicht mit Testpersonen abgestimmt.
- **Balancing und Playtests:** Lichtverbrauch, Torkosten und Sprungabstände sind rechnerisch und mit Testläufen abgestimmt, aber kaum von externen Personen gespielt worden. Schwierigkeit und Frustpunkte sind daher noch unsicher.
- **Lesbarkeit vs. Dunkelheit:** Das Konzept verlangt Dunkelheit, das Gameplay verlangt Lesbarkeit. Gegner und Gefahren mussten nachträglich aufgehellt werden. Ein durchgehendes Farbkonzept von Anfang an hätte das vermieden.
- **Technische Grenzen:** Lichtwerte liegen noch in GameState statt in Resources, „Deko verteilen“ überschreibt Hand-Deko, Kachelfugen zeigen das Raster, Forward+ verlangt Vulkan, Speicherstände fehlen.

Ausblick

Hochwertigere Soundeffekte, Speicherstände, Fähigkeits- und Gegnerdaten als Resources, weitere Bosse mit eigenen Mechaniken und Playtests mit der Zielgruppe.

4 Quellenverzeichnis

Quelle	Verwendung	Lizenz
Godot Engine 4.7 – godotengine.org	Engine, Editor, Laufzeit	MIT
Open Sans (Godot-Standardschrift)	Schrift in HUD und Menüs	SIL Open Font License 1.1
Hintergrundmusik (background_sound_1/2.ogg)	Musik in Menü und Leveln	[Quelle und Lizenz eintragen]
Soundeffekte	Selbst erzeugt (synthetisch generiert)	–
Grafik, Animationen, Shader, Level	Vollständig selbst erstellt; Kachel-Textur und Normal Map prozedural generiert	–
Ori and the Will of the Wisps (Moon Studios, 2020)	Inspiration für Spielgefühl, Movement und Bash-Mechanik; keine Assets, Figuren oder Namen übernommen	–
Claude (Anthropic), KI-Assistent	Unterstützung bei Konzept, Code und Dokumentation	–